CSE 574-D: Introduction to Machine Learning, Fall 2024

Dharaneeshwar Shrisai Kumaraguru – 50608064 – dshrisai@buffalo.edu

Assignment 3: Defining and Solving Reinforcement Learning Task

Part I: Define an RL Environment

Description of the Environment: Find Happiness

States: The environment is a 5x5 grid for finding Happiness, it has 25 unique states. Each state represents a position on the grid defined by a pair of coordinates (x, y).

Actions: The agent has 4 possible actions:

0 = Move Up

1 = Move Down

2 = Move Right

3 = Move Left

Rewards:

The agent receives a reward of +50 for reaching the goal state at (4, 4) and a sub goal at (3, 3): +20

Goal Reward: +50 🤩 Sub Goal: +20 😄

There are penalty states with negative rewards at: ☹️

(2, 2): -5

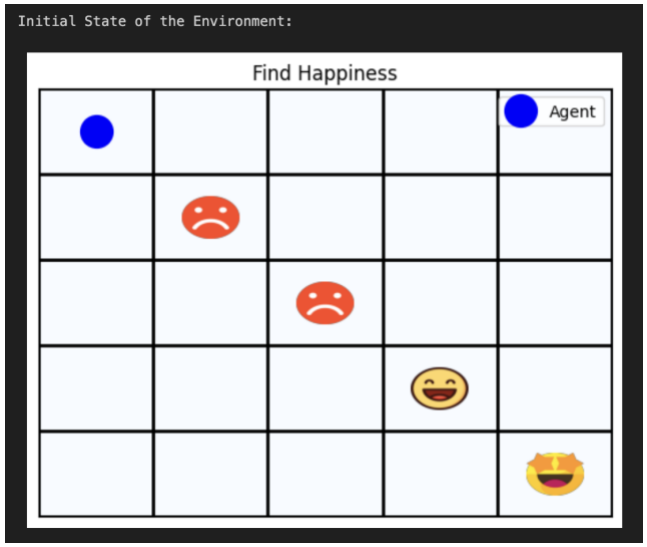
(1, 1): -3

The task is to navigate from the start position (0, 0) to the goal position (4, 4) while maximizing rewards and avoiding penalty states.

Visualization:

The environment includes a rendering function using matplotlib. It plots:

- The grid layout.
- Reward positions with relevant icons (awesome face for the goal, happy face for subgoal and sad face for negative rewards).
- The agent's position in the grid as a blue icon.



Safety in AI:

The environment ensures safety by:

- Restricting agent actions within the grid boundaries by making sure the agent's next state always remains with (0,0), (4,0), (0,4) & (4,4).
- Validating actions before execution to ensure they are within the defined action space of 0,1,2, & 3.
- Limiting rewards and penalties to predefined states, ensuring the agent's decisions are evaluated consistently. The reward is removed from the action state once the agent collects it.
- Providing clear termination conditions such as reaching the goal state.

Part II: Implement SARSA

Overview of the SARSA Method

The SARSA (State-Action-Reward-State-Action) algorithm is an on-policy reinforcement learning technique used for solving Markov Decision Processes (MDPs). It updates the Q-values (state-action values) by considering the action taken by the current policy and the expected reward from subsequent actions.

SARSA Update Rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha * [r + \gamma * Q(s',a') - Q(s,a)]$$

s: Current state

a: Current action

r: Reward obtained

s': Next state

a': Next action (chosen under the current policy)

α: Learning rate

γ: Discount factor

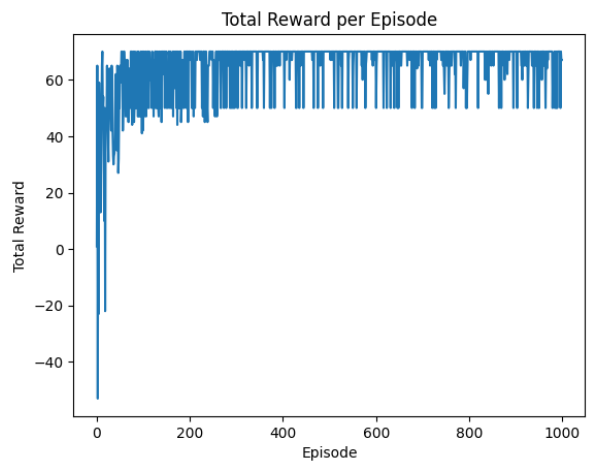
Key Features:

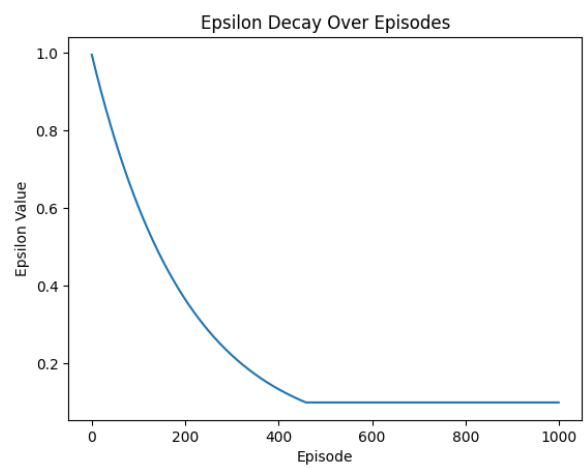
- SARSA updates Q-values based on actions that follow the current policy.
- Epsilon-greedy policy ensures exploration of suboptimal actions while prioritizing higher-reward actions over time.
- Temporal Credit Assignment: The algorithm effectively propagates rewards from future actions back to earlier states.

Trained Q table:

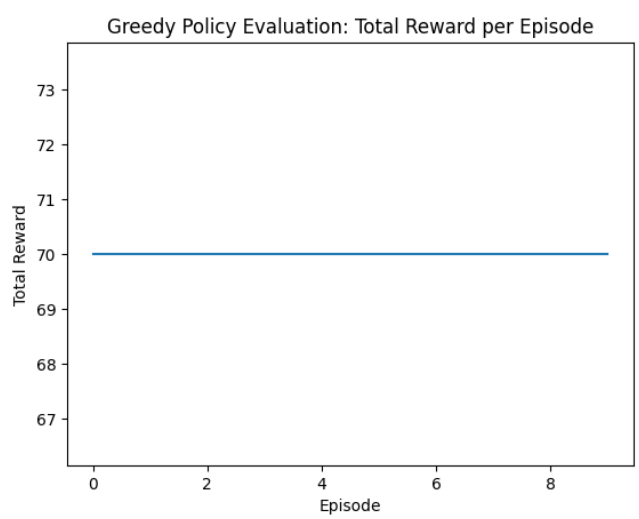
```
Trained Q-Table:
37.54411096490227       46.49205129631909       30.473304625352213      42.08090106312617
0.2269351483317843      10.793366699927596      0.5568711748225686      43.322421947400834
0.6684458230101773      0.36666776429108555     4.1819086890671855      -0.8151844906771067
0.847005233735181       1.9218683906102323      10.613711820185868      -0.41142683428543403
4.791938001980357       15.767490404913634      3.2204893467252185      2.1978907357060113
40.86526372541755       48.370489838196754      40.306195042381965      46.20547727354699
-1.1663044608292212     48.71707066644852       -0.12284109080168548    9.887341602637374
-0.3286937771564458     -2.769472300909348      10.097860318043534      -4.354705359776489
1.4349254791531498      1.6769867073814362      28.51149372515233       0.6828077307367799
4.554968737196862       38.297914408410165      9.119476450330904       2.901146966128282
46.45288723558804       46.97572593765792       44.6861075459   46.42220758818913
15.544288986033868      49.70362698086214       4.608673706430135       22.550116500651846
-0.13356228035530393    27.415621301017588      50.76372499951484       13.297790678750285
13.84735142721402       50.884907108161656      25.024025593558147      17.285324551386537
13.123016370400897      46.17390670261335       18.587952612747987      15.299935600149862
48.14224801152059       30.749775801386253      49.76169552925274       49.3523635045914
49.135152469357 39.527890631149035      53.920148491096704      47.18250781501784
39.89140027486305       47.44569477372697       61.9248347151057        49.26154931941179
45.51206692026858       49.39553699198747       47.89863459241269       48.26026092351673
26.339404725183808      49.999999898831156      39.73940941639667       28.406927001950585
4.4994987879285695      4.5577052806337655      39.55011910054843       0.10139417104543785
16.912716650772072      8.3928776092941 48.52158681421556       10.863499793820946
34.04887840427519       31.274760685422297      49.26288111281364       15.863903407775338
46.952900509956216      48.346190054441145      49.99999999999997       46.22446464065608
0.0     0.0     0.0     0.0
```

Total Reward per Episode plot & Epsilon Decay:





Greedy Policy Evaluation:



Advantages:

SARSA ensures updates are policy consistent. When applied in stochastic environments, SARSA avoids overly aggressive exploitation.

Disadvantages:

Exploration limits the speed of reaching optimal policies which tends to result in slower convergence. It is also sensitive to learning rate, discount factor, and exploration decay.

Influence of Hyperparameters

```
Testing Alpha=0.1, γ=0.9, Epsilon Decay=0.995, Reward=68.33
Testing Alpha=0.1, γ=0.9, Epsilon Decay=0.99, Reward=69.71
Testing Alpha=0.1, γ=0.9, Epsilon Decay=0.98, Reward=69.22
Testing Alpha=0.1, γ=0.95, Epsilon Decay=0.995, Reward=68.42
Testing Alpha=0.1, γ=0.95, Epsilon Decay=0.99, Reward=68.01
Testing Alpha=0.1, γ=0.95, Epsilon Decay=0.98, Reward=69.47
Testing Alpha=0.1, γ=0.99, Epsilon Decay=0.995, Reward=65.92
Testing Alpha=0.1, γ=0.99, Epsilon Decay=0.99, Reward=66.89
Testing Alpha=0.1, γ=0.99, Epsilon Decay=0.98, Reward=68.34
Testing Alpha=0.01, γ=0.9, Epsilon Decay=0.995, Reward=69.22
Testing Alpha=0.01, γ=0.9, Epsilon Decay=0.99, Reward=69.35
Testing Alpha=0.01, γ=0.9, Epsilon Decay=0.98, Reward=68.5
Testing Alpha=0.01, γ=0.95, Epsilon Decay=0.995, Reward=68.93
Testing Alpha=0.01, γ=0.95, Epsilon Decay=0.99, Reward=68.79
Testing Alpha=0.01, γ=0.95, Epsilon Decay=0.98, Reward=67.66
Testing Alpha=0.01, γ=0.99, Epsilon Decay=0.995, Reward=68.41
Testing Alpha=0.01, γ=0.99, Epsilon Decay=0.99, Reward=67.87
Testing Alpha=0.01, γ=0.99, Epsilon Decay=0.98, Reward=68.74
Testing Alpha=0.001, γ=0.9, Epsilon Decay=0.995, Reward=67.83
Testing Alpha=0.001, γ=0.9, Epsilon Decay=0.99, Reward=68.34
Testing Alpha=0.001, γ=0.9, Epsilon Decay=0.98, Reward=69.27
Testing Alpha=0.001, γ=0.95, Epsilon Decay=0.995, Reward=68.89
Testing Alpha=0.001, γ=0.95, Epsilon Decay=0.99, Reward=68.74
Testing Alpha=0.001, γ=0.95, Epsilon Decay=0.98, Reward=68.56
Testing Alpha=0.001, γ=0.99, Epsilon Decay=0.995, Reward=68.22
Testing Alpha=0.001, γ=0.99, Epsilon Decay=0.99, Reward=69.29
Testing Alpha=0.001, γ=0.99, Epsilon Decay=0.98, Reward=59.61
Best Parameters:
Aplha=0.1, γ=0.9, Epsilon Decay=0.99 (Average Reward: 69.71)
```

Hyperparameters significantly impact the performance and convergence of SARSA:

Learning Rate (α):

High values speed up learning but can cause instability. Low values ensure smoother updates but slow convergence.

Discount Factor (γ):

High values emphasize long-term rewards, suitable for problems with delayed goals. Low values prioritize immediate rewards, leading to shortsighted policies.

Epsilon Decay:

Controls exploration; faster decay encourages exploitation but risks premature convergence to suboptimal policies. Slower decay ensures extensive exploration but delays learning optimal behaviors.

Suggested Hyperparameter Values:

Based on the gridworld setup, the most efficient values for the problem were:

Learning Rate (α): 0.1

Discount Factor (γ): 0.9

Epsilon Decay: 0.99

These parameters resulted in a good balance between exploration, policy optimization, and reward maximization.

Best Results: Description, Plots, and Evaluation

After hyperparameter optimization, the setup with α=0.1, γ=0.9, and Epsilon Decay=0.99 yielded the best results.

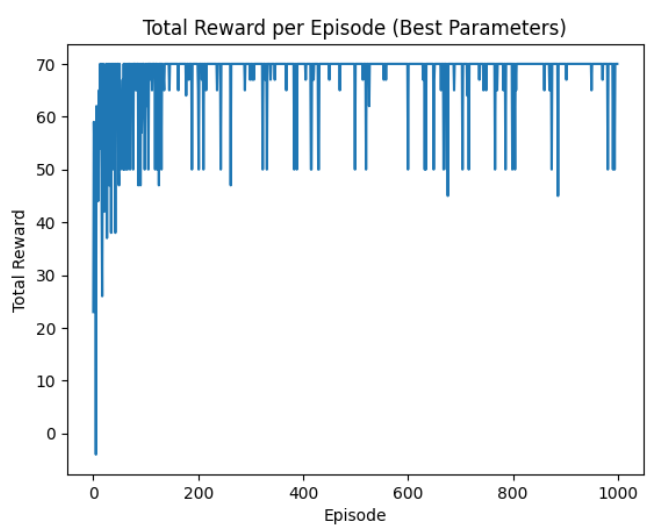
Performance Results:

Average Reward Over Last 100 Episodes: Consistently high, reflecting near-optimal policy.

Convergence: Q-values stabilized, showing convergence to the optimal policy.

Plots:

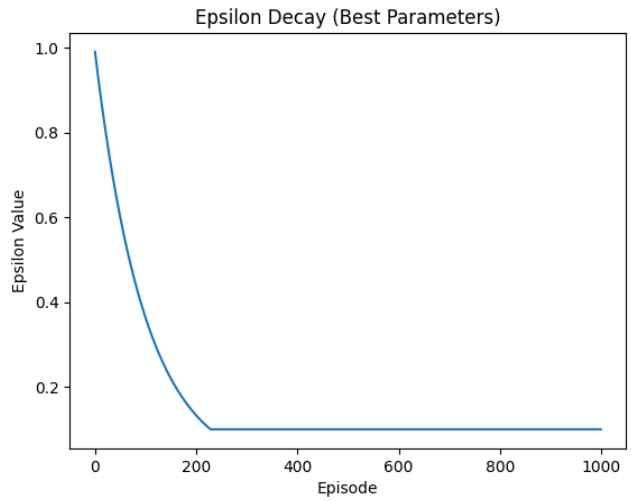
1. Total Reward per Episode:



Demonstrates increasing rewards over episodes. Reflects learning progress and effective policy optimization.

2. Epsilon Decay Over Episodes:

Shows a steady decrease, balancing exploration and exploitation effectively.



3. Greedy Policy Evaluation:

When tested with the trained Q-table, greedy policies consistently achieved high rewards across multiple episodes.

Trained Q-Table:

The trained Q-table below highlights the learned state-action values. The values near the goal state are higher, indicating optimal reward-seeking behavior.

21.343334833757524	29.229253944070514	5.458389329161086	22.257636013821514
-0.27630821270200273	-2.8318512931781936	11.249029460762426	1.5106919431945032
0.11229532751001528	2.455841267103484	20.319552382405085	0.06614260701458113
1.4002247797027052	31.583282230689544	0.1503204544172704	1.4463853532576703
0.06894524786916022	2.530761661467716	0.028870504905451323	-0.029969076728707134
19.000289052992837	32.45987695646883	6.484286358227097	27.519992087217503
0.8069221603493468	-0.3337358771151295	17.617410999275634	-0.39183505936552937
0.004561056600156883	1.0113147209702018	30.731504090000115	0.033840743643974935
4.058590177667943	42.47012398659183	3.7181071647568613	2.432233679356762
0.07903040694556229	20.936365080584636	2.272362197389982	0.009848918724618226
24.311764252635502	34.57950692700652	20.34810849576397	30.004505519920855
-0.5258611918431031	42.85822103322186	-3.5033280646528135	11.25382784960922
0.42384121116376466	42.88693455342966	2.106565671554684	3.5780170953808583
7.740679329088695	49.757347590891676	20.239927116379995	-2.120285084233794
2.134571914629169	44.190474004576 21	11.788936437397805	3.353249666532311
28.42026941898001	16.345151729520524	40.95368655994578	33.13655310038548
29.151632506437082	19.127341597151716	46.888649878718	34.29087677975488
24.56085314291735	30.64323940496863	47.07046343420921	37.4004511268498 16
41.643108835602824	42.280398888196885	44.11558102782137	43.518513711940365
31.534043608841074	49.99999999999997	41.670197561418306	35.30797860266096
30.67636431055539	0.11553230842487808	0.04370573665907592	0.5547143336448173
0.33376275747828327	2.1157992221501467	31.25401867395441	0.8552657007333105
9.424027609746027	12.9100021429716	42.01056055569573	3.8020219015774535
20.28881299490994	13.190983725109772	49.99903183701054	18.129771294012695
0.0 0.0 0.0	0.0		

Analysis

The SARSA algorithm successfully solved the grid world problem. The combination of $\gamma=0.9$ and slow epsilon decay (0.99) ensured long-term rewards and extensive exploration early in training. The rewards increased consistently across episodes, showing that the agent effectively learned to navigate the environment and avoid negative rewards.

Challenges:

- High penalties for specific states occasionally led to suboptimal paths during exploration.
- Convergence speed was slower due to the extensive exploration phase.
- Future Recommendations:
- Scaling: Introduce larger environments to test scalability.
- Alternative Policies: Compare with Q-learning to explore faster convergence.
- SARSA proved to be an effective, albeit slower, learning method for achieving consistent policies in the HappinessEnv setup.

<u>Part II: Implement N-step Double Q-learning</u>

Double Q-learning is an enhancement to the standard Q-learning algorithm that addresses the issue of overestimation bias in action-value estimates. In Q-learning, the action-value function is updated by selecting the maximum action-value from the Q-table, which tends to overestimate the value of actions due to noise and randomness in the environment.

Double Q-learning mitigates this bias by maintaining two separate Q-tables, QA and QB . At each time step:

- One Q-table is used to select the action using an epsilon-greedy policy.
- The other Q-table is used to evaluate the selected action and update the action-value.

Key Features of Double Q-Learning includes two Q-tables to reduce overestimation bias, Q-tables are updated alternately using each other's values to select and evaluate actions and Epsilon-greedy Policy to balance exploration and exploitation, the algorithm uses an epsilon-greedy policy for action selection.

Update Function:

If action a_t is selected using QA, the update is:

$$Q_A(s_t, a_t) \leftarrow Q_A(s_t, a_t) + \alpha\left(r_t + \gamma \max_{a'} Q_B\,(s_{t+1}, a') - Q_A(s_t, a_t)\right)$$

If action a_t is selected using QB, the update is:

$$Q_B(s_t, a_t) \leftarrow Q_B(s_t, a_t) + \alpha\left(r_t + \gamma \max_{a'} Q_A\,(s_{t+1}, a') - Q_B(s_t, a_t)\right)$$

Where α is the learning rate, γ is the discount factor, and r_t is the reward received after taking action a_t .

Advantages:

- Reduces Overestimation Bias: By maintaining two Q-tables, it reduces the bias in value estimation compared to standard Q-learning.
- Improved Stability: By alternating between QA and QB, it leads to more stable learning.

Disadvantages:

- Increased Memory Requirement: Storing two Q-tables requires more memory than standard Q-learning.
- Complexity: The alternating update logic increases the complexity of the algorithm and may slow down convergence.

2. Derivation of n-Step Double Q-Learning Update Rule

Let Q1(s,a) and Q2(s,a) represent the two Q-tables for state s and action a. The update rule for Double Q-learning can be broken down into the following steps:

1. Select action a_t at time t using an epsilon-greedy policy based on Q1 or Q2.
2. Evaluate the action a_t using the other Q-table.
3. Update the Q-tables using the reward and the estimated value of the next state-action pair.

The update rule for **n-step Double Q-learning** generalizes the Q-learning update rule by incorporating information from n steps of experience rather than a single step. The update process involves accumulating rewards over n steps, and then updating the Q-tables at time t+n using the n-step return.

$S_t = \{s_t, s_{t+1}, \ldots, s_{t+n-1}\}$ denotes the sequence of states from time step t to t+n−1.

$A_t = \{a_t, a_{t+1}, \ldots, a_{t+n-1}\}$ denotes the sequence of actions taken from time step t to t+n−1.

$R_t = \{r_t, r_{t+1}, \ldots, r_{t+n-1}\}$ denotes the sequence of rewards received during these n steps.

The n-Step Return

The n-step return G_t^n is computed, which is the cumulative reward from the current state for n time steps, discounted by γ:

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q_1\,(s_{t+n}, a') \quad \text{(if the action was selected based on Q2)}$$

or

$$G_t^{(n)} = \sum_{k=0}^{n-1} \gamma^k r_{t+k} + \gamma^n \max_{a'} Q_2\,(s_{t+n}, a') \quad \text{(if the action was selected based on Q1)}$$

Generalized Update Rule for Different n-Values

For n=1, the update rule corresponds to traditional Q-learning, where we update the Q-table after a single step. For larger n, the agent accumulates rewards over multiple steps before updating the Q-values.

1-Step

For n=1, the update is the same as standard Q-learning:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha\left(r_t + \gamma \max_{a'} Q_2\,(s_{t+1}, a') - Q_1(s_t, a_t)\right)$$

or

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha\left(r_t + \gamma \max_{a'} Q_1\,(s_{t+1}, a') - Q_2(s_t, a_t)\right)$$

2-Step

For n=2, the update involves considering the return over two steps:

$$G_t^{(2)} = r_t + \gamma r_{t+1} + \gamma^2 \max_{a'} Q_1\,(s_{t+2}, a')$$

The update is then:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha\left(G_t^{(2)} - Q_1(s_t, a_t)\right)$$

or

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha\left(G_t^{(2)} - Q_2(s_t, a_t)\right)$$

3-Step

For n=3, the return involves three steps:

$$G_t^{(3)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 \max_{a'} Q_1\left(s_{t+3}, a'\right)$$

The update becomes:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha\left(G_t^{(3)} - Q_1(s_t, a_t)\right)$$

or

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha\left(G_t^{(3)} - Q_2(s_t, a_t)\right)$$

4-Step

For n=4, the return involves four steps:

$$G_t^{(4)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \gamma^4 \max_{a'} Q_1\left(s_{t+4}, a'\right)$$

The update rule becomes:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha\left(G_t^{(4)} - Q_1(s_t, a_t)\right)$$

or

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha\left(G_t^{(4)} - Q_2(s_t, a_t)\right)$$

5-Step

For n=5, the return involves five steps:

$$G_t^{(5)} = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \gamma^4 r_{t+4} + \gamma^5 \max_{a'} Q_1\left(s_{t+5}, a'\right)$$

The update rule becomes:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha\left(G_t^{(5)} - Q_1(s_t, a_t)\right)$$

or

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha\left(G_t^{(5)} - Q_2(s_t, a_t)\right)$$

General Update Rule

The generalized n-step update rule for Double Q-learning is:

$$Q_1(s_t, a_t) \leftarrow Q_1(s_t, a_t) + \alpha\left(G_t^{(n)} - Q_1(s_t, a_t)\right)$$

or

$$Q_2(s_t, a_t) \leftarrow Q_2(s_t, a_t) + \alpha\left(G_t^{(n)} - Q_2(s_t, a_t)\right)$$

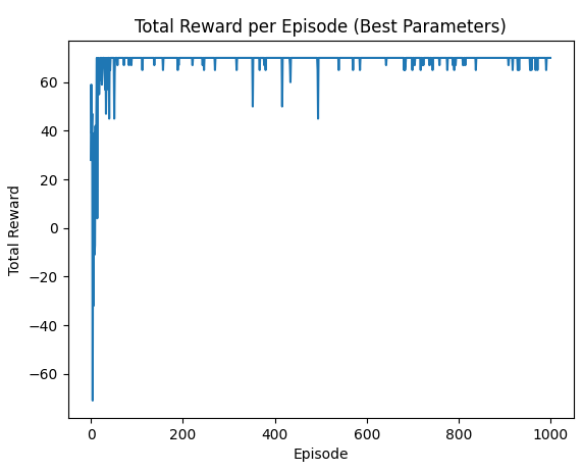
3. Hyperparameters and Their Influence on Performance

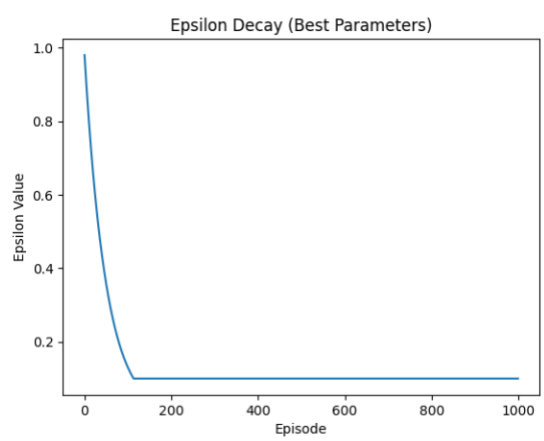
- Alpha (α): The learning rate. A higher value allows for faster learning but may lead to overshooting optimal values. A lower value may slow down convergence.
- Gamma (γ): The discount factor. A high value (γ→1) means the agent cares more about future rewards, while a low value (γ→0) prioritizes immediate rewards.
- Epsilon (ϵ): The exploration rate. A high epsilon encourages more random actions, and a low epsilon encourages selecting the action with the highest Q-value.
- Epsilon Decay: Controls how quickly the exploration rate decreases over time.
- n (n-step returns): The number of steps considered for the return. Larger n values take longer-term rewards into account, which can be beneficial in some environments but may also introduce more variance.

Optimal Hyperparameters: From empirical results, for this specific setup, the best performing parameters are:

- Alpha: 0.1
- Gamma: 0.95
- Epsilon Decay: 0.98

```
Testing Alpha=0.1, γ=0.9, Epsilon Decay=0.995, Reward=68.99
Testing Alpha=0.1, γ=0.9, Epsilon Decay=0.99, Reward=69.18
Testing Alpha=0.1, γ=0.9, Epsilon Decay=0.98, Reward=69.37
Testing Alpha=0.1, γ=0.95, Epsilon Decay=0.995, Reward=68.13
Testing Alpha=0.1, γ=0.95, Epsilon Decay=0.99, Reward=69.22
Testing Alpha=0.1, γ=0.95, Epsilon Decay=0.98, Reward=69.86
Testing Alpha=0.1, γ=0.99, Epsilon Decay=0.995, Reward=61.91
Testing Alpha=0.1, γ=0.99, Epsilon Decay=0.99, Reward=64.0
Testing Alpha=0.1, γ=0.99, Epsilon Decay=0.98, Reward=69.76
Testing Alpha=0.01, γ=0.9, Epsilon Decay=0.995, Reward=68.19
Testing Alpha=0.01, γ=0.9, Epsilon Decay=0.99, Reward=69.42
Testing Alpha=0.01, γ=0.9, Epsilon Decay=0.98, Reward=69.74
Testing Alpha=0.01, γ=0.95, Epsilon Decay=0.995, Reward=68.74
Testing Alpha=0.01, γ=0.95, Epsilon Decay=0.99, Reward=69.71
Testing Alpha=0.01, γ=0.95, Epsilon Decay=0.98, Reward=69.29
Testing Alpha=0.01, γ=0.99, Epsilon Decay=0.995, Reward=69.19
Testing Alpha=0.01, γ=0.99, Epsilon Decay=0.99, Reward=69.86
Testing Alpha=0.01, γ=0.99, Epsilon Decay=0.98, Reward=69.42
Testing Alpha=0.001, γ=0.9, Epsilon Decay=0.995, Reward=67.8
Testing Alpha=0.001, γ=0.9, Epsilon Decay=0.99, Reward=69.69
Testing Alpha=0.001, γ=0.9, Epsilon Decay=0.98, Reward=69.82
Testing Alpha=0.001, γ=0.95, Epsilon Decay=0.995, Reward=69.02
Testing Alpha=0.001, γ=0.95, Epsilon Decay=0.99, Reward=68.34
Testing Alpha=0.001, γ=0.95, Epsilon Decay=0.98, Reward=59.59
Testing Alpha=0.001, γ=0.99, Epsilon Decay=0.995, Reward=68.77
Testing Alpha=0.001, γ=0.99, Epsilon Decay=0.99, Reward=68.97
Testing Alpha=0.001, γ=0.99, Epsilon Decay=0.98, Reward=69.68
Best Parameters:
Alpha=0.1, γ=0.95, Epsilon Decay=0.98 (Average Reward: 69.86)
```



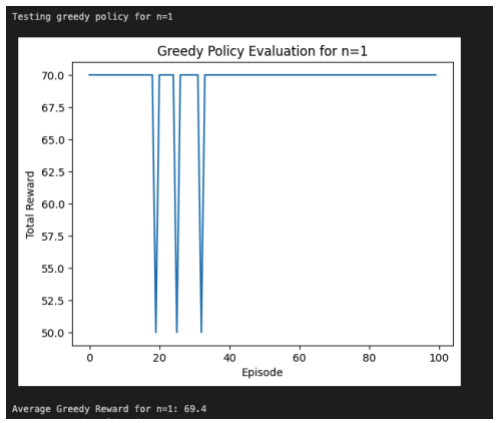


4. Results for Different Values of n

We experimented with various values of n (1, 2, 3, 4, 5) for n-step Double Q-learning. The results show that as n decreases, the agent considers longer-term rewards, which helps improve performance in environments with a delayed reward structure. However, too large n may lead to higher variance and slower convergence.

Optimal n:

- In our experiment, n=1 showed the best balance between learning speed and stability.



5. Comparison Between SARSA and n-Step Double Q-learning

SARSA (State-Action-Reward-State-Action) and n-step Double Q-learning are both model-free reinforcement learning algorithms, but they differ in how they update their Q-values.

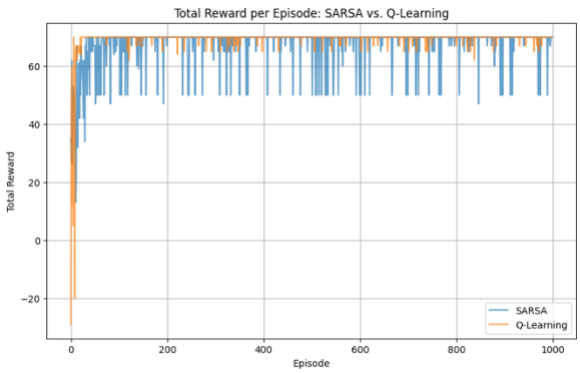
- SARSA uses the current policy to both select and evaluate the next action, leading to a more conservative learning approach.
- n-step Double Q-learning alternates between two Q-tables, reducing overestimation bias by using different tables for action selection and evaluation.

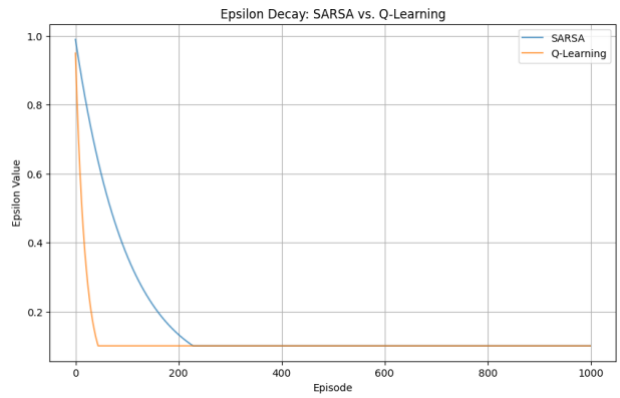
Performance Comparison:

- SARSA tends to perform well in environments where the optimal policy is not drastically different from the current policy and exploration does not need to be aggressive.
- n-step Double Q-learning outperforms SARSA in environments with delayed rewards or high variance, as it accounts for longer-term rewards and reduces overestimation bias.

Graph Interpretation:

- We observe that n-step Double Q-learning converges faster and with higher total rewards compared to SARSA. This is due to its better handling of delayed rewards and the reduction of bias from overestimation.





References:

https://ubuffalo-my.sharepoint.com/:f:/g/personal/avereshc_buffalo_edu/EoYTlYh-G0VAl58j9BZjX4YBgeZtS6kRsdKs1aG8Edt3og?e=W3E1mC
https://ubuffalo-my.sharepoint.com/:f:/g/personal/avereshc_buffalo_edu/EnzumGRMoUDjHB6m7qxcqYB3lS8vrLaK8Sd_MoY3-LnNA?e=Genopf
https://www.flaticon.com/free-icon/star_2584606?term=happy&page=1&position=16&origin=search&related_id=2584606
https://www.flaticon.com/free-icon/happy_6637188?term=happy&page=1&position=8&origin=search&related_id=6637188
https://www.flaticon.com/free-icon/face_14025917?term=sad&page=1&position=19&origin=search&related_id=14025917
http://incompleteideas.net/book/first/ebook/node78.html
https://ai.stackexchange.com/questions/22966/how-can-i-derive-n-step-off-policy-temporal-difference-formula